

Report on News Summarization Assessment Task

This report explains the key decisions, challenges, and future plans for the news article summarizer project. The system was built to pull information from web articles, create summaries, and check its own work.

Design Choices

The project was built in separate, modular parts to make it easy to test and improve each piece.

- **Context Retrieval Implementation** To find the right information to answer a user's question, we used a method called Dense Passage Retrieval. First, the article is broken down into small chunks of sentences. We chose to split by sentences because it is a simple and effective method for news articles. While tools like LangChain's RecursiveCharacterTextSplitter are powerful, they are more complex and better suited for structured documents like code and often not recommended for plain news text. After chunking, we turn the user's question and each text chunk into number vectors (embeddings). We then use cosine similarity to find the top 3 chunks that are most similar to the question. This method was chosen because it proves to be effective at understanding the meaning and context of the question, not just matching keywords. It provides a strong and efficient baseline for answering a specific query from a single document.
- **Language Model (LLM) Selection** We used the gpt-4.1-nano model for all text generation tasks. This model was chosen because it offers a good mix of performance, speed, and cost for our needs.
- **Reliable JSON Generation** The self-critique step needed to output data in a consistent JSON format. To make sure this worked every time, we used LangChain's built in structured output feature. We defined the exact structure we needed using a Pydantic model, which forces the LLM to return a correctly formatted JSON.
- **Self-Critique Mechanism** To check the quality of the generated summary, we used a second, separate LLM call. This AI was given the role of a "fact checker" to judge if the summary was faithful to the original text and relevant to the user's question. We chose this method because it provides a detailed, reasoned evaluation. A simpler technique, like comparing embeddings with cosine similarity, would only tell us if the text is similar and may suffer with fact checking nuances. For example, "Ahmad played Olympics in 2023" and "Ahmad played Olympics in 2024" would have larger

similarity score even though they are not quite the same thing. So, It cannot check for factual correctness or understand if the user's specific question was truly answered.

Challenges

A key challenge was finding a reliable way to automatically evaluate the quality of the generated summaries. It is difficult to programmatically determine if a summary is factually correct and truly answers a user's question. Our solution was to design the self-critique mechanism, where a separate AI model acts as an impartial judge. This "AI checking AI" approach proved to be an effective way to add a layer of quality control to the system.

Future Improvements

This project is a successful proof of concept, and several steps can be taken to make it ready for production.

- **Improved Scraping:** The web scraper works well but could be made even more robust for complex multi-source web scrapping that verifies the references mentioned. We could add more advanced parsing logic to handle a wider variety of non-standard website layouts and better extract article content.
- **Scalable Deployment:** Currently, the application is packaged in a Docker container. From now, we can work on deployment of this product. Moving the application to a managed cloud service like AWS or Google Cloud would provide auto-scaling, better security, and load balancing to handle more users.